Calculatrices interdites. Sur votre copie, les questions doivent apparaître dans l'ordre du sujet. Pensez à numéroter les pages et à donner les signatures des fonctions.

Graphes planaires

Le sujet est composé de quatre parties. Les parties 2 et 3 sont indépendantes et utilisent les notations de la partie 1. La partie 4 utilise les résultats des trois parties précédentes.

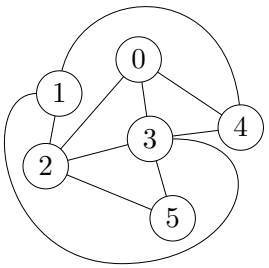

Figure 1

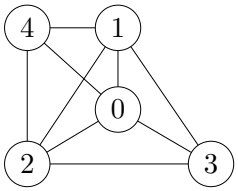

Figure 2

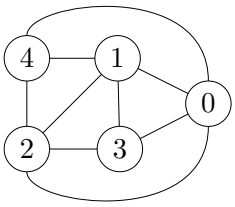

Figure 3

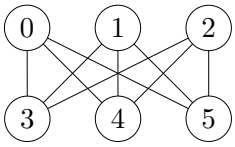

Figure 4

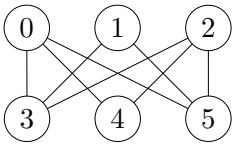

Figure 5

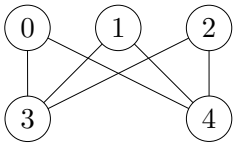

Figure 6

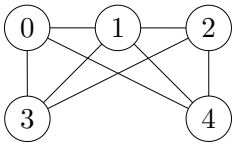

Figure 7

1 Préambule

Tous les graphes considérés sont non orientés et sans boucle.

1. (a) Donner la définition formelle de graphe non orienté.
   (b) Donner la définition de boucle.

Dans tout le sujet, les sommets d'un graphe G avec n sommets sont systématiquement étiquetés par $0, 1, 2, \ldots, n-1$; chaque entier $i \in [\![0, n-1]\!]$ est donc l'étiquette d'un et un seul sommet de G.
En Python, un graphe est représenté par une liste `(G: list[list[int]])` telle que `G[i]` est la liste d'adjacence du sommet d'étiquette `i` (cette liste ne contient jamais d'élément en double). Par exemple, le graphe de la figure 1 correspond à :

```
G_fig1 = [[2,3,4], [2,3,4], [0,1,3,5], [0,1,2,4,5], [0,1,3], [2,3]]
```

2. Donner les listes `G_fig2` et `G_fig3` pour représenter les graphes des figures 2 et 3.

Dans la suite, le type `list[list[int]]` sera noté `graphe`. Ainsi une fonction `(f(G: graphe) -> int)` est une fonction qui prend en entrée un graphe représenté par une liste `G` de type `list[list[int]]`, et qui renvoie un entier.

Définition 1 (informelle)**.** Un graphe G est dit ***planaire*** s'il peut être être dessiné dans le plan sans que deux arêtes ne se croisent.

Par exemple, le graphe de la figure 1 est planaire alors que le graphe de la figure 4 n'est pas planaire. De plus, contrairement aux apparences, le graphe de la figure 2 est planaire. En effet, les figures 2 et 3 représentent le même graphe. Puisque le graphe de la figure 3 est planaire, le graphe de la figure 2 l'est aussi.

3. Montrer que le graphe de la figure 5 est planaire.

Remarque. Définir formellement la notion de graphe planaire nécessite des outils avancés de topologie. Dans la suite, on se contentera de la définition informelle donnée ci-dessus. Il serait aussi possible d'utiliser comme définition la caractérisation du théorème 5 de la partie 4.

2 Transformations de graphes

2.1 Opérations sur des listes d'entiers

4. Écrire une fonction `suppr(L: list[int], i0: int) -> list[int]` qui renvoie une liste contenant les mêmes éléments que `L` où `i0` a été supprimé. Par exemple, avec `L = [4,5,1,4,9]` et `i0 = 4`, on obtient `[5,1,9]`. Votre fonction ne doit pas modifier la liste donnée en entrée.
5. Écrire une fonction `decaler(L: list[int], i0: int) -> NoneType` qui modifie la liste `L` en remplaçant tous les éléments `i > i0` par `i-1`. Par exemple avec `L = [5,1,4,9]` et `i0 = 4`, on obtient `[4,1,4,8]`. Votre fonction doit modifier la liste en entrée et renvoyer `None`.
6. Écrire une fonction `suppr_doublons(L: list[int]) -> list[int]` qui renvoie une liste contenant les mêmes éléments que `L` sans élément en double. Par exemple si `L` vaut `[4,8,2,8,3,6,4,0,3,4]`, on doit obtenir `[4,8,2,3,6,0]`. Votre fonction ne doit pas modifier la liste donnée en entrée et doit s'exécuter en temps $\Theta(n)$ où n est la taille de `L`.

2.2 Opérations sur des graphes

Dans cette partie, on s'intéresse à trois opérations :
– **L'opération de suppression d'une arête**. Par exemple, dans le graphe de la figure 4, on note s_1 et s_4 les sommets d'étiquettes 1 et 4. Alors, supprimer l'arête $\{s_1, s_4\}$ donne le graphe de la figure 5.
– **L'opération de suppression d'un sommet**. Soit $G_1 = (S, A)$ un graphe avec $n \in \mathbb{N}^*$ sommets. Supprimer un sommet $s_0 \in S$ d'étiquette $i_0 \in [\![0, n-1]\!]$ consiste à construire un graphe G_2 où le sommet s_0 a été supprimé ainsi que toutes les arêtes adjacentes à s_0. Afin que l'ensemble des étiquettes de G_2 soit égal à $[\![0, n-2]\!]$, un sommet d'étiquette $i > i_0$ dans G_1 est ré-étiqueté par $i-1$ dans G_2.
Par exemple, supprimer le sommet d'étiquette $i_0 = 4$ dans le graphe de la figure 4 donne le graphe de la figure 6. Notez qu'en particulier, le sommet d'étiquette 5 de la figure 4 vérifie $5 > i_0$ et est donc ré-étiqueté par 4 dans la figure 6.
– **L'opération de contraction d'une arête**. Soit $G_1 = (S, A)$ un graphe avec $n \geqslant 2$ sommets et s_1, s_2 deux sommets d'étiquettes i_1, i_2. Contracter l'arête $\{s_1, s_2\}$ consiste à construire un nouveau graphe G_2 où les sommets s_1 et s_2 ont été remplacés par un nouveau sommet t. Les voisins de t sont tous les voisins de s_1 et tous les voisins de s_2 (sauf s_1 et s_2 eux-mêmes). Afin que l'ensemble des étiquettes dans G_2 soit égal à $[\![0, n-2]\!]$, le sommet t est étiqueté par $\min(i_1, i_2)$ et un sommet d'étiquette $i > \max(i_1, i_2)$ dans G_1 est ré-étiqueté par $i-1$ dans G_2.
Par exemple, soient s_1 et s_4 les sommets d'étiquettes 1 et 4 dans la figure 4. Alors, contracter l'arête $\{s_1, s_4\}$ donne le graphe de la figure 7.

7. On considère un graphe G_1 représenté par une liste `(G1: graphe)`. Soient s_0, s_1, s_2 trois sommets de G_1 dont les étiquettes sont `i0, i1, i2`. Les fonctions écrites dans les questions qui suivent doivent renvoyer une nouvelle liste `G2` et ne pas modifier `G1`. De plus, vous devez faire en sorte que les sous-listes de `G2` ne contiennent pas d'élément en double (c'est déjà le cas pour les sous-listes de `G1`). Pensez à utiliser les fonctions de la partie précédente.
   (a) Écrire une fonction `suppr_arete(G1: graphe, i1: int, i2: int) -> graphe` qui renvoie la liste `(G2: graphe)` représentant le graphe obtenu après suppression de l'arête $\{s_1, s_2\}$. On pourra supposer sans le vérifier que $\{s_1, s_2\}$ est bien une arête de G_1.
   (b) Écrire une fonction `suppr_sommet(G1: graphe, i0: int) -> graphe` qui renvoie la liste `G2` représentant le graphe obtenu après suppression du sommet s_0.
   (c) Écrire une fonction `contr_arete(G1: graphe, i1: int, i2: int) -> graphe` qui renvoie la liste `G2` représentant le graphe obtenu après contraction de l'arête $\{s_1, s_2\}$. On pourra supposer sans le vérifier que $\{s_1, s_2\}$ est bien une arête de G_1.

3 Graphes complets

3.1 Cas des graphes non-bipartis

8. (a) Rappeler la définition d'un graphe complet.
   (b) Écrire une fonction `make_complet(n: int) -> graphe` qui renvoie un graphe complet à n sommets.
9. On rappelle que les graphes considérés dans ce sujet sont sans boucle et que les listes d'adjacence ne contiennent pas d'élément en double.
   (a) Écrire une fonction `est_complet(G: graphe, n: int) -> bool` qui indique si le graphe donné en entrée est complet avec `n` sommets. Votre fonction devra être de complexité $\mathcal{O}$`(n)`. On expliquera rapidement le choix d'implémentation.
   (b) Montrer que votre fonction est bien de complexité $\mathcal{O}$`(n)`.

3.2 Cas des graphes bipartis

Définition 2. Un graphe $G = (S, A)$ est dit ***biparti*** si S peut être décomposé en deux ensembles S_1, S_2 tels que toute arête possède une extrémité dans S_1 et l'autre dans S_2. Formellement, les deux ensembles S_1 et S_2 doivent vérifier :

$$S_1 \cap S_2 = \varnothing, \qquad\qquad S_1 \cup S_2 = S,$$

et pour toute arête $\{u_1, u_2\} \in A$, on doit avoir ($u_1 \in S_1$ et $u_2 \in S_2$) ou bien ($u_1 \in S_2$ et $u_2 \in S_1$).

Par exemple les graphes des figures 4 et 5 sont bipartis puisque les ensembles $S_1 = \{0, 1, 2\}$ et $S_2 = \{3, 4, 5\}$ conviennent. De même, le graphe de la figure 6 est biparti.

10. Montrer que le graphe de la figure 1 n'est pas biparti.

Définition 3. Un graphe est dit ***biparti complet*** si c'est un graphe biparti et que chaque sommet de S_1 est voisin de tous les sommets de S_2.

Par exemple le graphe de la figure 4 est biparti complet avec $|S_1| = 3$ et $|S_2| = 3$.

11. Écrire une fonction `make_biparti_complet` qui prend en entrée deux entiers $(n_1, n_2) \in \mathbb{N}^2$, et renvoie un graphe biparti complet où $|S_1| = n_1$ et $|S_2| = n_2$.

3.3 Reconnaissance des graphes bipartis complets

Soit G un graphe avec n sommets représenté par une liste `(G: graphe)`. Dans la suite, on souhaite écrire une fonction Python qui indique si G est biparti et s'il est complet. Pour cela, on va utiliser des parcours en largeur.

12. Soit s_0 un sommet de G d'étiquette `(i0: int)`.
    Écrire une fonction `parc_larg(G: graphe, i0: int) -> list[int]` qui renvoie la liste des sommets visités lors d'un parcours en largeur de G à partir de s_0. On pourra utiliser les files à double extrémités du module `collections` dont voici un exemple d'utilisation :

```
import collections
### Création d'une file à double extrémités (FDE)
L = collections.deque([5,7,3]) # L vaut [5,7,3]
print(len(L))                  # Affiche 3
### Ajout au début de la FDE
L.appendleft(8)                # L vaut [8,5,7,3]
### Ajout à la fin de la FDE
L.append(0)                    # L vaut [8,5,7,3,0]
### Suppression au début de la FDE
a = L.popleft()                # a vaut 8 --- L vaut [5,7,3,0]
### Suppression à la fin de la FDE
b = L.pop()                    # b vaut 0 --- L vaut [5,7,3]
```

La fonction `parc_larg` renvoie une liste qui ne contient que les sommets accessibles depuis s_0. Afin d'établir la liste de tous les sommets du graphe, on effectue plusieurs parcours en largeur jusqu'à ce que tous les sommets aient été atteints. Pour cela, on exécute la procédure suivante :

- On initialise une liste `(V: list[bool])` de taille n ne contenant que des `False`. Le booléen `V[i]` indique si le sommet d'étiquette `i` a déjà été visité.
- Pour chaque `i` $\in [\![$`0; n - 1`$]\!]$:
  - Si `V[i]` vaut `True`, on ne fait rien.
  - Sinon, on effectue le parcours en largeur de G à partir du sommet d'étiquette `i`. Pour chaque sommet d'étiquette `j` dans ce parcours, on remplace `V[j]` par `True`.
- On renvoie la concaténation de tous les parcours en largeur effectués.

13. Écrire une fonction `parc_larg2(G: graphe) -> list[int]` qui renvoie la concaténation des parcours en largeur comme décrit ci-dessus.

Supposons que G est un graphe biparti. Soient S_1 et S_2 les deux ensembles donnés par la définition 2. En Python, S_1 va être représenté par une liste de booléens `(S1: list[bool])` de taille n telle que `S1[i]` vaut `True` si le sommet d'étiquette `i` est un élément de S_1 et `False` sinon. De même, S_2 va être représenté par une liste de booléens `(S2: list[bool])`.

14. Écrire une fonction `decomp_biparti(G: graphe) -> (list[bool] * list[bool]) ou NoneType` qui renvoie les listes `S1` et `S2` ou bien `None` si G n'est pas biparti (notez que les listes `S1` et `S2` ne sont pas uniques). Pour cela on utilisera la liste `P` renvoyée par l'appel à `parc_larg2(G)`.
15. En déduire une fonction `est_biparti_complet(G: graphe, n1: int, n2: int) -> bool` qui indique si le graphe donné en entrée est un graphe biparti complet avec $|S_1| = n_1$ et $|S_2| = n_2$. Dans le cas où le nombre de sommets du graphe est incompatible avec n_1 et n_2, votre fonction s'exécutera en temps constant.

4 Caractérisation des graphes planaires

Définition 4.
⋆ On appelle ***transformations*** les trois opérations vues dans la partie 2 (suppression d'une arête, suppression d'un sommet et contraction d'une arête).
⋆ Soient $k \in \mathbb{N}$ et G_1, G_2 deux graphes. On dit que G_2 est un ***mineur d'ordre*** k de G_1, si G_2 peut être obtenu à partir de G_1 en appliquant une suite de k transformations.
⋆ Soient G_1, G_2 deux graphes. On dit que G_2 est un ***mineur*** de G_1 s'il existe $k \in \mathbb{N}$ tel que G_2 est un mineur d'ordre k de G_1.
⋆ On note K_1 le graphe complet à 5 sommets et K_2 le graphe biparti complet de la figure 4

Théorème 5 (admis)**.** *Un graphe $G = (S, A)$ est planaire si et seulement si l'ensemble des mineurs de G ne contient ni K_1 ni K_2.*

16. À l'aide du théorème 5, répondre aux questions suivantes :
    (a) Soit $n \in \mathbb{N}$ et G un graphe complet à n sommet. Le graphe G est-il planaire ?
    (b) Soit $n_1 \in \mathbb{N}$, $n_2 \in \mathbb{N}$ et G le graphe biparti complet avec $|S_1| = n_1$ et $|S_2| = n_2$. Le graphe G est-il planaire ?

Pour tester si un graphe est planaire, on va générer tous ses mineurs et tester si K_1 ou K_2 en font partie.

17. (a) Écrire une fonction `mineurs_ordre1(G: graphe) -> list[graphe]` qui renvoie une liste contenant tous les mineurs d'ordre 1 du graphe.
    (b) En déduire une fonction `est_planaire(G: graphe) -> bool` qui indique si le graphe donné en entrée est planaire.

Remarque. L'algorithme présenté ici est particulièrement inefficace. En 1972, l'informaticien Robert Tarjan (prix Turing 1986) a publié dans sa thèse un algorithme de complexité linéaire en n permettant de tester si un graphe est planaire.